

Zeta’s Asynchronous Model: A Compiler-Driven Approach to Safe and Performant Concurrency

Eyad Amr

<https://github.com/Voxon-Development/zeta-lang>

May 22, 2026

Abstract

In this article, we explore Zeta’s asynchronous model and how it: avoids *async infection* as found in Rust and JavaScript; upholds high performance and throughput guarantees; introduces safety features for asynchronous programming such as scheduler-misuse detection and suspension-hazard analysis; enables the compiler to interact with the asynchronous runtime for suspension; remains independent of any specific runtime or async model (unlike Go, where roughly 90% of the time one is reasoning in terms of goroutines, or JavaScript and Dart, where one is reasoning in terms of `async/await`); and, in some senses, improves ergonomics over `async/await`—especially relative to Rust.

Contents

1 Related Topics Before Zeta’s Asynchronous Model

First of all, Zeta’s asynchronous model is not a runtime nor an async model. It is a compiler feature that allows one to write asynchronous code in a way that is safe and performant.

To truly understand Zeta’s asynchronous model, we must first understand why we need an asynchronous model in the first place.

1.1 Why Do We Need an Asynchronous Model?

It is not as flashy as a new memory-safety model or a new language feature, but it is the part of the language that will be used the most when one needs to scale. It is the reason entire languages are popular (such as Go and, partially, Rust), and it powers almost all of the most successful services today (such as Twitter, Netflix, and Stripe) and almost anything that needs to scale.

To know what the problem really is, we must understand the problem itself. We begin with Go and its scheduler, and explain why it is not a good fit for Zeta’s systematic approach.

1.2 Go

Go's scheduler is built around three core components:

- **G (Goroutine):** a lightweight task.
- **M (Machine):** an OS thread.
- **P (Processor):** an execution context that schedules goroutines.

Go uses an **M:N scheduler**, meaning that many goroutines are multiplexed across fewer OS threads while utilizing multiple CPU cores.

Each P has a **local run queue (LRQ)** for scheduled goroutines, while a **global run queue (GRQ)** stores unscheduled work. An M must be attached to a P to execute goroutines, and only one M can run on a given P at a time.

The runtime maintains up to **GOMAXPROCS** processors (Ps), typically based on the number of logical CPUs available. When a local queue becomes empty, work is pulled from the global queue or stolen from other processors.

1.2.1 Why Is This Important?

Because Zeta's **standard-library opt-in asynchronous runtime** is biased toward stackful coroutines such as **green threads** (e.g., **goroutines**). What Go's model is missing, however, is the safety aspect:

- *Suspension hazards:* What happens if a lock is held and suspension occurs?
- *Scheduler misuse:* What if one blocks on a goroutine? Prior to Go 1.14, CPU-bound goroutines could fully starve the scheduler since goroutines were only preempted at function-call boundaries. Go 1.14 introduced asynchronous preemption via signals, which mitigates this—but blocking on a goroutine and holding locks across suspension remain real hazards regardless.
- *Affinity issues:* Thread migration, i.e. moving from thread A to thread B within the same goroutine.
- *Runtime assumptions:* Thread-local storage, OS resources, and execution context.

1.2.2 Example: Hidden Assumptions

Many systems assume:

- thread-local storage is stable;
- OS resources are thread-bound;
- execution context is consistent.

Go breaks these assumptions at times. A goroutine may:

- start on thread A;
- suspend;
- resume on thread B.

Why Is This a Problem? Go does provide `runtime.LockOSThread()` for cases where thread identity genuinely matters—CGo, OS-level namespaces, thread-bound OS resources. However, this is an opt-in escape hatch, not a guarantee. The default behavior is migration, meaning any code that implicitly assumes thread-local identity without pinning is silently broken.

Go has also suffered from scheduler misuse: blocking synchronously inside a goroutine, or holding a mutex across a channel operation, can produce deadlocks that neither the compiler nor the runtime detects. (Before Go 1.14, tight CPU loops could additionally starve other goroutines entirely; asynchronous preemption fixed that specific case, but the lock and channel hazards remain.)

It has also suffered from:

- goroutine leaks that heavily increase memory usage and led to the existence of dynamic detection tools;
- unsafe usage found in 24% of 2,438 popular Go projects, motivated primarily by OS/C interop but also by performance optimization—with a complementary study finding that 91% of projects transitively depend on at least one unsafe usage;
 - See also: *Uncovering the Hidden Dangers: Finding Unsafe Go Code in the Wild* (arXiv:2010.11242).

A great many of these issues are caused by a goroutine waiting on a channel that is never closed, a lock that is never unlocked, or a context that is not cancelled properly on error paths.

Is this not a memory-safety issue? Yes, but not entirely. Consider the following example:

```
1 package main
2
3 import (
4     "net/http"
5     "sync"
6 )
7
8 var mu sync.Mutex
9 var jobs = make(chan int)
10 var results = make(chan int)
11
12 var sharedCounter int
13
14 func worker() {
15     for job := range jobs {
16         // worker needs lock too
17         mu.Lock()
18         sharedCounter += job
19         mu.Unlock()
20
21         results <- sharedCounter
22     }
23 }
24
25 func handler(w http.ResponseWriter, r *http.Request) {
26     mu.Lock()
27
28     // simulate request data
29     job := 1
30
31     // send job to worker
32     jobs <- job
33
34     // wait for result
```

```

35     res := <-results
36
37     mu.Unlock()
38
39     w.Write([]byte("result"))
40     _ = res
41 }
42
43 func main() {
44     go worker()
45
46     http.HandleFunc("/", handler)
47     http.ListenAndServe(":8080", nil)
48 }

```

Listing 1: A Go HTTP handler with a hidden deadlock cycle.

This looks fine, does it not? At first glance:

- the mutex protects shared state;
- the worker processes jobs;
- the channel separates concerns;
- the HTTP handler is simple.

1.2.3 The Deadlock Scenario (Real Execution)

An HTTP request arrives and the handler grabs the `mu` lock immediately. It then tries to send the job into the channel; if the worker is busy or the buffer is full, the send simply blocks. So the handler ends up holding the lock while waiting on results.

The worker wakes up and pulls the job from the channel. It then tries to lock `mu`, but the handler still has it. This forms a cycle in which each side waits on the other.

The actual deadlock is simple: one thread holds the lock and waits on the results channel; the other has the job from the channel and waits on the lock.

Cycle: `mu` → `results` → `jobs` → `mu`.

1.3 Rust

We have established why Go’s scheduler is not a good fit for Zeta’s systematic approach: technically the runtime is close to what we want, but the compiler is not.

This is where Rust shines, because it has one of the most sophisticated compilers in the world that is also mainstream.

How Does Rust Handle Async? Rust approaches the same problem from the opposite direction:

Instead of hiding execution behavior in the runtime, Rust pushes it into the type system.

Rust does not have built-in async execution at the runtime level. Instead, `async` is a compiler transformation into state machines (`Future` types), executed by an external runtime ([Tokio](#), [async-std](#), etc.).

The core idea is that when one writes:

```
1 async fn fetch() -> u32 {  
2     42  
3 }
```

Listing 2: A trivial Rust async function.

Rust transforms it into something conceptually equivalent to a state machine that implements the `Future` trait and is manually polled by an executor (the most common executor being `Tokio`). So instead of “running asynchronously,” Rust functions become lazy computations that can be resumed after suspension points.

Suspension Is Explicit (`.await`). Unlike Go, where suspension is implicit in runtime operations, Rust forces explicit suspension:

```
1 let data = fetch().await;
```

Each `.await` is a potential suspension point—a compiler-marked state-transition boundary *and* an explicit yield to the executor.

1.3.1 Why Is This Important?

This means that Rust’s async is not just about concurrency, but also about safety while preserving performance. Rust prevents some concurrency issues, such as:

1. Memory Safety Across Suspension. Rust’s borrow checker prevents many classes of invalid memory access across `.await` points—dangling references, invalid stack-frame assumptions, and use-after-free in async tasks are largely caught at compile time.

However, this comes with significant complexity. Self-referential async state machines (a struct holding a reference into itself) require `Pin<P>` to be safe, since moving the struct would invalidate the internal pointer. The `Pin` API is notoriously difficult to use correctly and is one of the sharpest edges in the Rust async model. Additionally, the official async book notes that [calling a blocking function from an async context, or implementing `Future` incorrectly, can silently pass both the compiler and unit tests](#)—so “memory safe” does not mean “all async bugs are caught at compile time.”

2. Some Classes of Data Races. Rust uses the borrow checker to manage shared mutable state through explicit synchronization with `Arc<Mutex<T>>`. Unsafe concurrent mutations are rejected at compile time, eliminating many race conditions early.

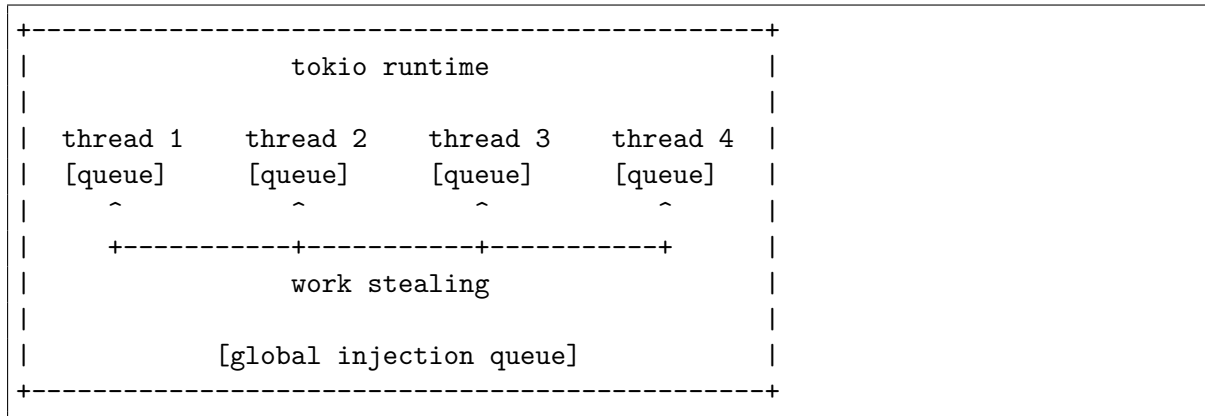
1.4 How Rust’s “Runtime” Works

Rust is different from most languages because it **does not** ship with an async runtime. This is intentional. The language provides `async/await` syntax and the `Future` trait, but *something* must actually poll those futures to completion.

Let us examine how the most popular executor works: Tokio. (There are many others, such as `async-std`, `embassy`, and `smol`, but we focus specifically on Tokio.)

Tokio comprises four core components.

(1) A Multithreaded Task Scheduler (Work-Stealing). It works something like the following diagram:



Each worker thread has a local run queue: a fixed-size ring buffer of 256 tasks. When one calls `tokio::spawn`, the task goes onto the current worker’s local queue. If that queue is full, half of the tasks are moved to a shared injection queue.

When a worker finishes its current task, it checks its local queue first. If the local queue is empty, it then checks the injection queue. If that is empty too, it tries to steal work from another worker’s local queue. This process is called *work-stealing*.

Thus, `tokio::spawn` is essentially saying “schedule this task to be polled by one of the worker threads.” A single worker thread may be juggling thousands of tasks, but each task costs only a few hundred bytes and a function-pointer swap. That is why one can have millions of concurrent tasks on a single machine.

Each task is a state machine implementing the `Future` trait:

```

1 // what tokio sees every task as
2 trait Future {
3     type Output;
4     fn poll(self: Pin<&mut Self>, cx: &mut Context) -> Poll<Self::Output>;
5 }
6
7 // Poll::Pending = not done yet, waker registered, come back later
8 // Poll::Ready(v) = done, here is the value

```

Listing 3: The shape of a task as seen by Tokio.

The driver thread handles I/O readiness via `epoll/kqueue/IOCP` and wakes tasks when their I/O is ready.

(2) An Async I/O Driver Built on `epoll/kqueue/IOCP`. This is integrated with the multithreaded task scheduler and lives in a driver thread.

The driver thread handles I/O readiness via `epoll/kqueue/IOCP` and wakes tasks when their I/O is ready, as follows:

```

task calls TcpStream::read().await
    -> registers waker with epoll fd
    -> returns Poll::Pending
    -> task removed from thread queue

epoll fires when data arrives
    -> waker.wake() called
    -> task re-queued on a thread
    -> thread polls the future again
    -> Poll::Ready(data) returned

```

A task calls `TcpStream::read().await`, which performs the following:

- Registers the waker with the `epoll` file descriptor.
- Returns `Poll::Pending` and the task is removed from the thread queue.

Then `epoll` fires when data arrives, and the following events are processed in order:

- `waker.wake()` is called;
- the task is re-queued on a thread;
- the thread polls the future again;
- the future finally returns `Poll::Ready(data)`.

1.4.1 Why Is This Not Enough?

Well, if it were enough, this article would not exist—so what is really missing?

Zeta is inspired by this level of sophistication, but Rust’s safety still does not prevent some logic bugs, and it comes with trade-offs.

Async Infection. Async constraints can propagate upward through the architecture. This forces more of the system to become asynchronous than might conceptually be needed. It feels like a bigger issue than just syntax.

I am not entirely certain, but it appears that this structural coupling comes from the async model itself.

1.4.2 A Naive Example

```

1 pub async fn read_file(file: &str) -> Result<String, std::io::Error> {
2     // async file IO
3     tokio::fs::read_to_string(file).await
4 }
5
6 fn load_config() {
7     let config = read_file("config.txt").await;
8 }

```

We can simply add `async` to `load_config`, but the issue is deeper:

The Real Problem: Infection of Call Hierarchy. Once a single low-level operation becomes async:

- file I/O becomes async;
- config loading becomes async;
- initialization becomes async;
- service bootstrap becomes async;
- framework entry points become async.

Eventually the entire program shape is dictated by whether a primitive operation is async or not.

Why This Becomes a System-Level Problem. In real systems, this leads to:

- API pollution (everything becomes async even if CPU-bound);
- runtime-dependency locking (one cannot easily mix sync and async code);
- architecture distortion (design follows executor constraints, not domain logic);
- hidden coupling to the runtime model (Tokio, I am looking at you).

No Way to Analyze Lock and Suspension Behavior. Once locks and async coexist, suspension propagates into critical sections: it can be held across `.await` suspension points, channels can wait inside suspension points, and dependency inversion across tasks becomes possible. This is illustrated in the following example.

1.4.3 Real-World Example of a Deadlock in Rust

To replicate an actual Rust issue, consider the following code, which can compile and run but silently deadlocks:

```
1 use axum::{
2     extract::State,
3     routing::get,
4     Router,
5 };
6 use std::sync::Arc;
7 use tokio::{
8     sync::{Mutex, mpsc},
9     time::{sleep, Duration},
10 };
11
12 #[derive(Clone)]
13 struct AppState {
14     counter: Arc<Mutex<u64>>,
15     tx: mpsc::Sender<Job>,
16 }
17
18 struct Job {
19     value: u64,
20     respond: mpsc::Sender<u64>,
21 }
22
23 #[tokio::main]
```



```

24 async fn main() {
25     let counter = Arc::new(Mutex::new(0));
26
27     let (tx, rx) = mpsc::channel::<Job>(100);
28
29     let state = AppState {
30         counter: counter.clone(),
31         tx,
32     };
33
34     tokio::spawn(worker(rx, counter));
35
36     let app = Router::new()
37         .route("/", get(handler))
38         .route("/update", get(update))
39         .with_state(state);
40
41     axum::serve(
42         tokio::net::TcpListener::bind("127.0.0.1:3000")
43             .await
44             .unwrap(),
45         app,
46     )
47     .await
48     .unwrap();
49 }
50
51 async fn worker(
52     mut rx: mpsc::Receiver<Job>,
53     state: Arc<Mutex<u64>>,
54 ) {
55     while let Some(job) = rx.recv().await {
56
57         let mut guard = state.lock().await;
58
59         *guard += job.value;
60
61         sleep(Duration::from_millis(200)).await;
62
63         let result = *guard;
64
65         let _ = job.respond.send(result).await;
66
67         // lock dropped implicitly
68     }
69 }
70
71 async fn handler(State(state): State<AppState>) -> String {
72     let (resp_tx, mut resp_rx) = mpsc::channel(1);
73
74     let mut guard = state.counter.lock().await;
75
76     let value = *guard;
77
78     state.tx.send(Job {
79         value,
80         respond: resp_tx,
81     }).await.unwrap();
82
83     // Wait for the response.. wait, the lock is actually still held
84     let result = resp_rx.recv().await.unwrap();
85     // We never release the lock, so the worker will never finish
86

```

```

87 // lock released here (too late)
88 *guard = result;
89
90 "ok".to_string()
91 }
92
93 async fn update(State(state): State<AppState>) -> String {
94     let mut guard = state.counter.lock().await;
95
96     sleep(Duration::from_millis(100)).await;
97
98     *guard += 10;
99
100    "updated".to_string()
101 }

```

Listing 4: A Rust HTTP server that compiles cleanly but silently deadlocks.

This is a classic actor-style HTTP pipeline:

- the Axum HTTP server receives requests;
- shared state (`counter`) is protected by a mutex;
- jobs are sent to a background worker via `mpsc`;
- the worker updates shared state and replies via a per-request channel.

Conceptually:

HTTP handler → job queue → worker → shared state → response channel → HTTP handler.

This is a request–response pipeline with shared mutable state.

This is a deadlock, but Rust will not catch it at compile time, and the explicitness of `async/await` infection did not save us. Let us see why:

```

1 while let Some(job) = rx.recv().await {
2     // Meaning: worker waits for jobs (a place where it can pause, suspending point #1)
3     let mut guard = state.lock().await;
4     // Meaning: worker gets a mutex, on shared state, ok
5     // worker keeps exclusive access to the global state, basically like "hands on the wheel"
6     *guard += job.value;
7     // modifies shared state safely, no argument there
8     sleep(Duration::from_millis(200)).await;
9 }

```

1.4.4 First Critical Issue

The first real trouble spot is *suspension while holding a mutex*.

Note: the code uses `tokio::sync::Mutex`, not `std::sync::Mutex`. This distinction matters. `std::sync::Mutex` held across an `.await` would cause a different problem—it is `!Send`, so the compiler typically rejects it in `async` tasks (or it blocks a thread if somehow forced). `tokio::sync::Mutex` is explicitly designed to be held across `.await`, which is why the compiler accepts this code with no complaint. The deadlock here is purely a logic bug—a circular dependency—not a misuse of the lock type. That is precisely what makes it invisible to the type system.

Even though this mutex is `async`-friendly, one still:

- holds the lock for too long;
- while other tasks start piling up.

It is *not* a deadlock “yet.”

```
1 let result = *guard;
2 // reads state while the lock is still technically held
3 let _ = job.respond.send(result).await;
```

1.4.5 Second Critical Issue

The worker is still “morally” in the critical section, even though the code looks casual, and it performs an `.await` during a channel send.

The worker pauses while still holding the lock. This is where we mess up the structure—that little pause and lock combination.

```
1 // request side dependency inversion in 'handler()'
2 let (resp_tx, mut resp_rx) = mpsc::channel(1);
3 // a response channel per request, this creates a synchronization dependency tied to a single
  request's lifetime
4
5 let mut guard = state.counter.lock().await;
```

1.4.6 Third Critical Issue

The handler now grabs the global mutex, so:

- the handler holds the counter lock.

```
1 let value = *guard; // this reads the counter value
2
3 state.tx.send(Job {
4     value,
5     respond: resp_tx,
6 }).await.unwrap();
```

At this moment, the handler is still holding the mutex and yet it sends a job to the worker.

The dependency chain therefore becomes:

handler holds lock → sends job → worker depends on that same lock.

```
1 let result = resp_rx.recv().await.unwrap();
```

1.4.7 Fourth Critical Issue

The handler now waits for the worker response, but it still holds the mutex. This creates a reverse dependency cycle, which is not great.

```
1 *guard = result;
2 // this updates the state, still inside the lock scope so nothing is really freed, not even a
  little.
```

```
1 let mut guard = state.counter.lock().await;
2 sleep(...).await;
3 *guard += 10;
```

The main problem comes from holding the lock across a suspension point, and the worker pattern exhibits the same structure. This raises the odds of contention and can also make scheduling inversion more likely in some cases.

From the dependency graph, this stands out as the key part. The handler locks the counter, sends the job, and then waits on the response while still keeping everything locked the whole time. On the worker side, it waits for the job first, then grabs the lock, performs the suspend and sleep step, and finally sends the response back. Connecting these two paths together is where things get messy.

The hidden cycle is: the handler holds the counter lock and waits for the worker response. The worker needs the counter lock and depends on scheduler progress, which may be stalled indirectly from handler pressure.

Congratulations: we have achieved a *★ deadlock in the safest mainstream language ★*.

1.4.8 Important Note About Rust

When it comes to the memory side of the async model, Rust is one of the best languages in mainstream usage at handling it.

One cannot easily have a memory-safety issue concurrently, but one can have a logic issue concurrently—and that is what this article is about.

The point is that stackless and stackful coroutine systems in Rust can *also* leak execution state: suspended tasks remain indefinitely alive due to missing completion, cancellation, or scheduling conditions, even in the absence of memory-safety violations.

1.5 So... What Now?

We have discussed the two most popular languages that exhibit these issues, and most other languages are usually a branch of one of them (C#/JS/Dart may face the same issues but without the safety that Rust brings).

Java really merits a closer look here. The callback approach one sees with `thenApply` and `thenAccept` comes from the `CompletableFuture` API (which is especially prominent prior to Project Loom).

When Project Loom arrived, it gave us virtual threads, so now one writes blocking code with regular methods—no need for awkward `async` or `await` keywords. Under the hood, Loom runs many of these lightweight threads (M:N scheduling), much like Go's goroutines. However, some old headaches remain. If one holds a platform lock while a virtual thread is suspended, one is in trouble. If a virtual thread waits forever on a channel that never closes, one has leaks. And if one grabs a `synchronized` block inside a virtual thread, one actually pins its carrier thread and loses the lightweight scheduling advantage. In effect, Go's headaches are still here in Loom—just wearing a different outfit.

Erlang takes its own approach to concurrency. On BEAM, each process has its own heap, and processes communicate by copying messages, not by sharing memory. This makes whole classes of memory bugs much harder; the structure of the runtime enforces it. Still, Erlang is not

bulletproof. One can run into protocol deadlocks (process A waits for B, and B waits for A), mailboxes filling up with unmatched messages, or spawning so many processes that everything bogs down. And if one reaches for NIFs (Native Implemented Functions), it is just as unsafe.

2 Zeta's Approach V1

Zeta takes a different approach. It cannot rely purely on stackful coroutines, and it cannot just use `async/await` due to both sets of issues described above. So instead of reasoning about a function based on whether it is `async` or not, as in the following example:

```
1 async fn read_file(name: &str) -> Result<String, std::io::Error> {
2     // ...
3 }
4
5 fn main() -> Result<(), std::io::Error> {
6     let content = read_file("file.txt").await; // Invalid without '#[tokio::main]' and 'async
7     println!("{content}");
8 }
```

we do not actually need to reason about whether a function is `async` or not. We can model it this way:

`suspend` is used and it models the execution-yield probability.

In other words:

`suspend` means the function *may* yield execution.

So let us write some equivalent Zeta code:

```
1 suspend fn read_file(name: &str) -> zeta::io::Error!String {
2     // ...
3 }
4
5 fn main() -> zeta::io::Error!void {
6     content := try await read_file("file.txt");
7     println(content);
8 }
```

This will compile, but why? How will a low-level language function properly when there is no `async/await` (which implies no hidden runtime)? One would be mistaken to assume that this is what Zeta wants to be. Let me explain.

Zeta is a low-level language. It does not want to hide the runtime; it wants to be transparent about it. However, as we discussed, `async/await` has issues even when it is “explicit.” To understand why `main` need not be `suspend`, let us study this code.

```
1 suspend fn read_file(name: &str) -> zeta::io::Error!String {
```

This means something different from the Rust example:

Define a function named `read_file` that takes a reference to a string which is encoded in binary (not heap-allocated) and that can return a `String` or an I/O error via `!` union contracts. The function `read_file` may suspend and yield execution to another thread, but it is *not* asynchronous.

If this is still unclear, let us examine what suspension really is.

2.1 Suspension

Suspension is the act of yielding execution to any other resource. It can be caused by:

- *Parking*¹ a stackful coroutine (for example, when `epoll` occurs in a goroutine, that goroutine is parked);
- Waiting on a channel;
- Sometimes lock contention;
- Cooperative scheduling (such as Java’s `Thread.yield()`);
- Async APIs such as `epoll` and `io_uring`.

The difference between suspension and blocking is that when a thread is suspended, it is not blocked: it is just temporarily stopped and can be resumed later. When a thread is blocked, by contrast, the scheduler can *reuse* the thread it is on.

This is what `.await` is in Rust, `await` in C#/JS/Dart, and what some schedulers do implicitly (such as in Java and Go).

2.2 What Is the Difference?

Finally, we reach the interesting part. After that explanation, we can assert something important:

Suspension $\neq$ Asynchronous Execution.

Suspension is the act of yielding execution to any other resource—but what if that “resource” is just the main thread? Then it simply blocks, especially if the main thread itself is not a worker thread (which is common in constrained environments).

Let us return to our main function in Zeta:

```
1 fn main() -> zeta::io::Error!void {  
2     content := try await read_file("file.txt");  
3     println(content);  
4 }
```

Did you notice something? `read_file` is `suspend`, but `main` is not. Because `read_file` may yield execution, but on the main thread, and the main thread is not a worker thread, there is no way to resume it—so it simply blocks.

This is great. It means we can have functions that work the same way without “duplicating” the code (or at least the signatures) for sync and async.

2.3 “I Have a Lot of Questions”

Let us address them.

2.3.1 “What About the Runtime? There Needs to Be a Runtime for Asynchronous Execution.”

The runtime should be a controlled, opt-in environment (similar to that of Rust).

¹Parking: temporarily stopping execution while preserving resumable state.

```

1 // Once this is started, it just starts a thread pool and executor,
2 // and offloads it to static memory, so if it was called AGAIN
3 // then it is almost entirely free
4 static let virtual_executor: VirtualExecutor = VirtualExecutor::start();
5
6 fn main() -> zeta::io::Error!void {
7     content := virtual_executor.spawn(suspend fn() -> void {
8         try await read_file("file.txt")
9     });
10    std::out.println(content.join());
11 }

```

For a low-level language, it feels wrong to hide executors entirely.

2.3.2 How Does This Interact with FFI Calls?

Bindings.

```

1 // epoll wrapper FFI binding
2 extern "C" {
3     fn epoll_create1(flags: i32) -> i32;
4     fn epoll_ctl(epfd: i32, op: i32, fd: i32, event: *EpollEvent) -> i32;
5     fn epoll_wait(epfd: i32, events: *EpollEvent, maxevents: i32, timeout: i32) -> i32;
6     fn io_uring_wait_cqe_raw(ring: *IoUring, cqe: *IoUringCqe) -> i32;
7 }
8
9 // The binding author takes responsibility here
10 // epoll_wait can block the thread, so suspend is explicit and honest
11 suspend fn epoll_wait_async(epfd: i32, events: *EpollEvent, maxevents: i32) ->
12     zeta::io::Error!i32 {
13     result := epoll_wait(epfd, events, maxevents, -1);
14     if result < 0 {
15         return zeta::io::Error.from_errno();
16     }
17     return result;
18 }
19
20 // io_uring submission, doesn't block, nosuspend is honest
21 nosuspend fn io_uring_submit(ring: *IoUring, sqe: *IoUringSqe) -> zeta::io::Error!void {
22     result := io_uring_submit_raw(ring, sqe);
23     if result < 0 {
24         return zeta::io::Error.from_errno();
25     }
26 }
27
28 // io_uring completion wait, can block waiting for completion
29 suspend fn io_uring_wait_cqe(ring: *IoUring) -> zeta::io::Error!*IoUringCqe {
30     unsafe {
31         cqe: *IoUringCqe = undefined;
32         result := io_uring_wait_cqe_raw(ring, &cqe);
33         if result < 0 {
34             return zeta::io::Error.from_errno();
35         }
36         return cqe;
37     }
38 }

```

2.3.3 If There Is No `async/await`, and No State Machines, Then What Does `await` Do?

Well, nothing—literally.

It is just an explicit suspension point, and it is not `async` propagation, because `suspend fn` functions can be used in plain `fn` contexts—`suspend` does not say the function is `async`. It only states that the function may yield execution to another resource if this (virtual) thread is controlled by the scheduler. By using `await`, one explicitly tells the scheduler that one acknowledges that this may `suspend`.

2.3.4 “What About Non-Suspend Functions Calling Suspend? How Are They Handled?”

They are transitively turned into `suspend` functions and inferred (just as almost all of one’s lifetimes in Rust are inferred):

```
1 suspend fn read_file(name: &str) -> zeta::io::Error!String {
2     // ...
3 }
4
5 // Tooling and compiler hint: Depends on suspending function
6 fn parse_config(name: &str) -> zeta::io::Error!JsonNode {
7     content := try await read_file(name);
8     // process content
9 }
10
11 // Tooling and compiler hint: Depends on suspending function
12 fn init_program() -> zeta::io::Error!void {
13     config := try await parse_config("config.json");
14     database_data := try await parse_config("database.json");
15     english_language := try await parse_config("en_US.json");
16     // process content
17 }
```

2.4 Dispatcher Approach and Its Integration with `suspend fn` Functions

Zeta’s executor is not a hidden scheduler like Go, Python, or JS. It is an explicit, compiler-known construct that declares its own capabilities as `const` statements. The compiler reads these declarations to reason about what is legal within any given execution context.

The executor is defined as a compiler-known interface:

```
1 @compiler::Known
2 interface Executor {
3     const CAN_SUSPEND: bool;
4     const IS_SINGLE_THREADED: bool;
5     const IS_REENTRANT: bool;
6     const MAX_CONCURRENCY: ?usize;
7
8     /// Initialize the executor
9     fn start();
10
11     /// Returns Future<T> where T is inferred from what the task returns
12     /// not awaited here, just scheduled
13     fn spawn<T>(task: suspend fn() -> T) -> Future<T>;
14 }
```



```

15  /// Blocks the current context until the task completes
16  /// used as the entry point, not for general use
17  fn block_on<T>(task: suspend fn() -> T) -> T;
18
19  /// Creates a scope where all spawned tasks must complete
20  /// before the scope exits, compiler enforces this
21  suspend fn scope<T>(body: suspend fn(scope: &TaskScope) -> T) -> T;
22 }
23
24 // Structured concurrency scope
25 // All tasks spawned here are children of this scope
26 // scope cannot exit until all children complete or are cancelled
27 @compiler::Known
28 struct TaskScope {
29     executor: &Executor,
30 }
31
32 impl TaskScope {
33     // Spawns a child task within this scope
34     // Future is scoped, it cannot outlive the scope
35     // compiler enforces this via lifetime-like rules
36     fn spawn<T>(this, task: suspend fn() -> T) -> ScopedFuture<T>;
37
38     // Waits for ALL tasks in the scope to complete
39     suspend fn join_all(this) -> void;
40
41     // Waits for the FIRST task to complete, cancels the rest
42     // useful for racing tasks against each other
43     suspend fn join_any(this) -> void;
44
45     // Cancels all tasks in this scope
46     nosuspend fn cancel_all(this) -> void;
47 }
48
49 // A future that cannot outlive its parent scope
50 // compiler errors if you try to store this outside the scope
51 struct ScopedFuture<T> {
52     inner: Future<T>,
53     _scope: &TaskScope,
54 }
55
56 impl<T> ScopedFuture<T> {
57     suspend fn get(this) -> T { ... }
58     @compiler::Pure
59     nosuspend fn is_done(this) -> bool { ... }
60 }

```

Listing 5: The compiler-known `Executor` interface and structured-concurrency scope.

The `@compiler::Known` attribute is what bridges user-space structs into the compiler’s reasoning engine. The compiler does not know about any specific executor type; it only knows the `Executor` trait and reads the `const` declarations from whatever implements it.

As an example, let us implement an `InterruptExecutor` (which is intentionally limited):

```

1  // Interrupt handler executor
2  // Suspension is illegal, nesting is possible
3  struct InterruptExecutor {}
4
5  impl Executor for InterruptExecutor {
6      const CAN_SUSPEND: bool = false;
7      const IS_SINGLE_THREADED: bool = true;
8      const IS_REENTRANT: bool = true;

```

```

9  const MAX_CONCURRENCY: ?usize = 1;
10
11  fn start(this) { ... }
12  // nosuspend propagates to tasks as well, compiler enforces this
13  nosuspend fn spawn(task: nosuspend fn() -> T) -> Future<T> { ... }
14  nosuspend fn block_on(task: nosuspend fn() -> void) -> void { ... }
15 }

```

Let us use it in our code:

```

1  static let interrupt_executor: InterruptExecutor = InterruptExecutor::init();
2
3  fn handle_interrupt() -> void {
4      // COMPILER ERROR: InterruptExecutor.CAN_SUSPEND = false
5      // but read_file is a suspend fn
6      content := interrupt_executor.spawn(nosuspend fn() -> void {
7          try await read_file("file.txt")
8      });
9
10     interrupt_executor.spawn(nosuspend fn() -> void {
11         single_threaded_task();
12     });
13 }
14
15 fn single_threaded_task() -> void {
16     // InterruptExecutor.IS_SINGLE_THREADED = true
17     // compiler can prove no other task runs concurrently here
18     // lock elision is valid, compiler may warn that this mutex is redundant
19     mutex := Mutex::new(0);
20     guard := try await mutex.lock();
21 }

```

How is this possible? Transitive suspension solves it. Because non-suspend functions can be called from suspend functions, the compiler takes over for this inference (which is straightforward and can be done during IR lowering), and the compiler can prove that no other task runs concurrently here.

But suspension is much broader than asynchronous execution in general, so when one wants to do something like this, the question arises:

2.4.1 Why Cannot We Just Do the Same with Traditional `async/await`?

Because in `async/await`, suspension is implied by syntax rather than declared as behavior. The compiler knows that a suspension point exists at every `await`, but it knows nothing about what that suspension *means*.

Consider the following example:

```

1  async fn do_work() {
2      let mutex = std::sync::Mutex::new(0);
3      let guard = mutex.lock().unwrap();
4
5      // Rust actually catches this one specific case
6      // but only because MutexGuard is !Send, not because
7      // it understands suspension semantics
8      some_io().await;
9
10     drop(guard);
11 }

```

Rust caught this and can reject it, yes, but only because the guard is `!Send`, not because it understands suspension semantics.

If it *were* `Send`:

```
1 async fn do_work() {
2     // tokio's mutex IS Send, so this compiles fine
3     // but you just held a mutex across a yield point
4     // in a single threaded executor, this is a deadlock
5     // in a multi threaded executor, this might be fine
6     // the compiler has absolutely no idea which situation you're in
7     let mutex = tokio::sync::Mutex::new(0);
8     let guard = mutex.lock().await;
9
10    some_io().await; // guard held across here, deadlock possible
11
12    drop(guard);
13 }
```

The deadlock potential of holding `guard` across `some_io().await` is completely different between these two cases. In the single-threaded case, it is almost certainly a deadlock. In the multi-threaded case, it might be fine. The compiler sees identical code in both cases and says nothing.

```
1 // You also can't reason about WHY something suspends
2 // all of these look identical to the compiler at the suspension level
3 some_io().await;           // suspending for I/O readiness
4 channel.recv().await;     // suspending waiting on a channel
5 mutex.lock().await;       // suspending on lock contention
6 tokio::task::yield_now().await; // explicitly yielding cooperatively
7
8 // And there's no way to declare "this function must never suspend"
9 // you can write nosuspend conceptually but the language has no way to enforce it
10 fn definitely_not_suspending() {
11     // nothing stops someone from calling this from async context
12     // and nothing stops this from calling async code via block_on
13     // which may panic at runtime if already inside a runtime
14     tokio::runtime::Handle::current().block_on(some_async_fn());
15 }
```

Additionally, even if we essentially just rename `suspend` to `async`, `suspend` is still more honest in general:

```
1 // suspend is honest
2 suspend fn read_file(name: &str) -> zeta::io::Error!String {
3     // I might yield, that's all I'm saying
4     // the dispatcher decides what that means
5     // the caller doesn't need to change
6     // no state machine, no future, no polling
7 }
8
9 // async would be a lie
10 async fn read_file(name: &str) -> zeta::io::Error!String {
11     // implies: I run concurrently
12     // implies: I return a Future<String> not a String
13     // implies: you need an executor to drive me
14     // implies: caller must be async too
15     // none of this is true in the model
16 }
```

Even if one manages to remove the necessity of `async` from `fn` functions, so that `async fn` functions can be used in `fn` functions, one has essentially recreated this model. In the best case

it is less honest in general (a lock suspension is not async, for example; trying to reason about it with the same model but with `suspend` named `async` would be misleading); in the worst case, one does not even have the same guarantees as this model.

2.5 Real-World Application of Zeta’s Async Model

Let us “rewrite” the Rust HTTP-server code in Zeta, assuming all the libraries exist. (This is a rough sketch and may not accurately reflect real Zeta syntax.)

```

1 struct AppState {
2     counter: Arc<VirtualMutex<u64>>,
3     tx: mpsc::Sender<Job>,
4 }
5
6 struct Job {
7     value: u64,
8     respond: mpsc::Sender<u64>,
9 }
10
11 static let virtual_executor: VirtualExecutor = VirtualExecutor::start();
12
13 fn main() -> zeta::io::Error!void {
14     counter := Arc::new(VirtualMutex::new(0));
15     (tx, rx) := mpsc::channel<Job>(100);
16
17     state := AppState {
18         counter: counter.clone(),
19         tx,
20     };
21
22     virtual_executor.spawn(suspend fn() -> void {
23         await worker(rx, counter);
24     });
25
26     app := Router::new()
27         .route("/", get(handler))
28         .route("/update", get(update))
29         .with_state(state);
30
31     try await axum::serve(
32         try await TcpListener::bind("127.0.0.1:3000"),
33         app,
34     );
35 }
36
37 suspend fn worker(
38     rx: mpsc::Receiver<Job>,
39     state: Arc<VirtualMutex<u64>>,
40 ) -> void {
41     while let Some(job) = try await rx.recv() {
42         guard := try await state.lock(); // acquires lock
43         *guard += job.value;
44         await sleep(Duration::from_millis(200));
45         result := *guard;
46         try await job.respond.send(result);
47         // guard dropped here
48     }
49 }
50
51 suspend fn handler(state: AppState) -> zeta::io::Error!String {
52     (resp_tx, resp_rx) := mpsc::channel(1);

```

```

53
54 // guard is acquired and held
55 guard := try await state.counter.lock();
56 value := *guard;
57
58 try await state.tx.send(Job {
59     value,
60     respond: resp_tx,
61 });
62
63 // COMPILER ERROR: suspension cycle detected
64 //
65 // handler holds: state.counter (VirtualMutex)
66 //   -> suspended at: resp_rx.recv()
67 //     waiting for: worker to send on job.respond
68 //
69 // worker:
70 //   -> suspended at: state.counter.lock()
71 //     waiting for: handler to release state.counter
72 //
73 // handler cannot resume until worker sends
74 // worker cannot resume until handler releases the lock
75 // neither can make progress
76 //
77 // suspension cycle: handler -> worker -> handler
78 result := try await resp_rx.recv();
79
80 *guard = result;
81 return "ok";
82 }
83
84 suspend fn update(state: AppState) -> zeta::io::Error!String {
85     guard := try await state.counter.lock();
86
87     // no cycle here, sleep does not depend on any lock
88     // held by another suspended task
89     // compiler sees no suspension cycle, this is fine
90     await sleep(Duration::from_millis(100));
91
92     *guard += 10;
93     return "updated";
94 }

```

Listing 6: Zeta sketch of the same HTTP-server pattern, with the cycle caught by the compiler.

This is almost the same example as the Rust code. The intentional deadlock is in `handler` and it would never compile. Let us break it down and see how the theoretical Zeta compiler would catch it.

2.5.1 Step 1: Lock-Liveness Tracking

The compiler tracks `guard` as a live lock on `state.counter` from acquisition to drop:

```

1 guard := try await state.counter.lock(); // lock live from here
2 // ...
3 result := try await resp_rx.recv();      // suspension point while lock is live
4 // ...
5 *guard = result;
6 // guard dropped here, lock released

```

`VirtualMutex.HOLD_ACROSS_SUSPEND = ALLOWED`, so holding it across a suspension point is not, in itself, an error. The compiler does not stop here.

2.5.2 Step 2: Suspension Dependency Graph

Because `await` is an explicit acknowledgement marker, the compiler knows exactly where each task suspends and on what it is waiting:

```
handler suspends at resp_rx.recv()
-> unblocked by: worker sending on job.respond
-> condition: worker must reach job.respond.send()

worker suspends at state.counter.lock()
-> unblocked by: handler releasing state.counter
-> condition: handler must drop guard
```

2.5.3 Step 3: Cycle Detection

Handler progress requires that the worker sends on `job.respond`; meanwhile, worker progress requires that the handler releases `state.counter`; and the handler will release the lock only after `resp_rx.recv()` returns, which returns only after the worker sends on `job.respond`.

This is how the cycle works.

2.5.4 Step 4: Why update Is Fine

```
1 suspend fn update(state: AppState) -> zeta::io::Error!String {
2   guard := try await state.counter.lock();
3   await sleep(Duration::from_millis(100)); // suspension point
4   *guard += 10;
5   return "updated";
6 }
```

2.6 Blocking

Blocking is fundamentally different from suspension. When a coroutine suspends, the scheduler can park it and reuse the thread for other tasks. When code blocks, the entire OS thread is parked and the scheduler cannot reuse it. This is invisible to the suspension model by default.

However, it can be modelled in the same way suspension is modelled. Blocking is simply a third behavioural declaration:

```
1 // may block the OS thread entirely
2 // scheduler cannot reuse this thread while blocked
3 blocking fn read_file_sync(name: &str) -> zeta::io::Error!String {
4   // synchronous syscall, parks the OS thread
5 }
6
7 // may suspend (yield cooperatively)
8 suspend fn read_file_async(name: &str) -> zeta::io::Error!String {
9   // async io_uring call, parks the coroutine not the thread
10 }
11
```

```

12 // guaranteed neither
13 nosuspend fn compute(x: u64) -> u64 {
14     // pure computation, no yielding, no blocking
15 }

```

The three-state model becomes:

nosuspend	= no yielding, no blocking, always makes forward progress
suspend	= may yield cooperatively, scheduler can reuse the thread
blocking	= may park the OS thread, scheduler cannot reuse it

The executor const system already has the vocabulary for this. CAN_BLOCK is added alongside CAN_SUSPEND:

```

1 @compiler::Known
2 interface Executor {
3     const CAN_SUSPEND: bool;
4     const CAN_BLOCK: bool;    // can tasks call blocking fns on this executor?
5     const IS_SINGLE_THREADED: bool;
6     const IS_REENTRANT: bool;
7     const MAX_CONCURRENCY: ?usize;
8     // ...
9 }
10
11 // VirtualExecutor should never have blocking calls on its worker threads
12 // blocking here starves all other tasks on that thread
13 struct VirtualExecutor {}
14
15 impl Executor for VirtualExecutor {
16     const CAN_SUSPEND: bool = true;
17     const CAN_BLOCK: bool = false;
18     // ...
19 }
20
21 // A dedicated thread pool for absorbing blocking calls
22 // similar conceptually to tokio's spawn_blocking, but explicit and typed
23 struct BlockingExecutor {}
24
25 impl Executor for BlockingExecutor {
26     const CAN_SUSPEND: bool = false;
27     const CAN_BLOCK: bool = true;
28     // ...
29 }

```

This enables the following class of errors to be caught:

```

1 static let virtual_executor: VirtualExecutor = VirtualExecutor::start();
2 static let blocking_executor: BlockingExecutor = BlockingExecutor::start();
3
4 suspend fn handler(state: AppState) -> zeta::io::Error!String {
5     // COMPILER ERROR: VirtualExecutor.CAN_BLOCK = false
6     // read_file_sync is a blocking fn
7     // this parks the OS thread and starves all other tasks on it
8     content := try await read_file_sync("config.txt");
9 }
10
11 suspend fn handler_correct(state: AppState) -> zeta::io::Error!String {
12     // FINE: offloaded to blocking_executor which declares CAN_BLOCK = true
13     // virtual executor thread is not starved
14     content := try await blocking_executor.block_on(blocking fn() -> zeta::io::Error!String {
15         try await read_file_sync("config.txt")
16     });
17 }

```

The `blocking` keyword is also inferrable transitively, just like `suspend`.

Because Zeta compiles to a specific target, the entire blocking/suspension chain from one's code down to the exact syscall is statically known. The tooling can show the full trace:

```
COMPILER ERROR: blocking call in VirtualExecutor context

handler() -> load_config() -> read_file() -> PlatformFileSystem::read()
                                         |
                                         LinuxFileSystem::read() [blocking]
                                         |
                                         syscall: read(fd, buf, count)

VirtualExecutor.CAN_BLOCK = false
Consider: PlatformFileSystem::read_async() or offload to BlockingExecutor
```

This is more precise than suspension analysis in one specific sense. Suspension behavior can vary at runtime depending on the dispatcher; blocking down to a syscall is completely deterministic given the target triple. `read()` on Linux x86_64 always parks the thread; no runtime variability is possible.

The remaining limitation is the same as with suspension: FFI calls that internally block are invisible to this analysis. The binding author is responsible for annotating honestly, and `unsafe` remains the escape hatch where they cannot.

2.7 Limitations of This Asynchronous Model

We have explained why this async model is great, but not yet its disadvantages (over modern research, and perhaps even current solutions).

2.7.1 Inapplicable for Stackless Coroutines (Unless Adjusted)

The language does not produce state machines; even though it has explicit `awaits`—which help the scheduler and compiler reason about it—it is just a regular function call that does not produce magic.

2.7.2 Has the Limitations of Stackful Coroutines

These can include memory overhead, etc.

2.7.3 Only Statically Analyzable Deadlocks and Cycles Can Be Proven

Consider this function pointer:

```
1 suspend fn handler(state: AppState) -> zeta::io::Error!String {
2   guard := try await state.counter.lock();
3
4   // cycle is through a function pointer or trait object
5   // compiler cannot see what this actually calls at runtime
6   try await (state.callback)();
```


One cannot know which pointer this resolves to, so the compiler would need to go out of its way either to ignore this or to execute it under `unsafe`.

2.7.4 Both Blocking and Suspension Are Platform-Dependent

Unlike reasoning about `async`, where one can slap `async` on anything that can suspend, the compiler is also reasoning about blocking, which requires more sophisticated compiler work and research. Additionally, not all APIs are purely suspending or purely blocking, and some can do both. For example:

```
futex(addr, FUTEX_WAIT, expected, timeout);
```

This syscall is not inherently “blocking” or “non-blocking” in isolation. Instead, it is state-dependent:

- If `*addr != expected`, it returns immediately without sleeping.
- If `*addr == expected`, it transitions the calling thread into kernel-managed sleep via the scheduler.
- If a signal or timeout occurs, it may abort before or during suspension.

So the same call can behave as:

- a fast no-op return,
- a full thread suspension, or
- a partially completed wait that is cancelled before suspension stabilizes.

This makes it fundamentally different from a pure asynchronous suspension point or a pure blocking syscall. The behaviour is determined dynamically by memory state at the exact moment of the kernel check, not by the API shape itself, which can be problematic for this model—and we **need** to **research** how to **solve** something like this.

An alternative solution (which can work as a workaround) is to have some sort of annotation such as `@UnknownSuspensionCondition` that hints to the compiler: “Hey, this will probably suspend, but we do not know how it will work; it may park the stackful coroutine, the whole OS thread, or nothing.”

2.7.5 FFI and stdlib Should Be Disciplined About the First suspend and blocking Points

If `async` is not used in the standard libraries or in programs that do not depend on the standard library, then the compiler is blind to it.

Similarly, the compiler is blind to the `async`-safety and guarantees coming from FFI calls (just as Rust is blind to the memory guarantees coming from FFI calls). We must therefore be as disciplined as we were in the FFI examples shown previously, and both the standard library and non-stdlib-dependent binaries and libraries should be disciplined.

2.8 The End

This is how Zeta will track suspensions. We have established that:

- No state machines are implemented (Zeta is biased toward **stackful** coroutines), which may improve compile times and debuggability;
- The performance is as fast as the scheduler will allow (the compiler does not produce any magic);
- The code can scale, as one does not need to duplicate the code;
- The code can be safely validated by the compiler directly, with minimal additions to lowering and the type system, without a separate (dynamic or static) external tool such as in Rust and Go;
- Not only can suspending code be reasoned about transitively, but so can code that may block, given smarter (standard) library engineering.

2.8.1 Sources

- [Diving Deep into Goroutines.](#)
- [The main source code of the Go scheduler.](#)
- Bob Nystrom, *What Color Is Your Function?* (2015).
- *Uncovering the Hidden Dangers: Finding Unsafe Go Code in the Wild* (arXiv:2010.11242).
- [Go 1.14 Release Notes—Asynchronous Preemption.](#)
- [The State of Asynchronous Rust—The Async Book.](#)
- [Futures and the Async Syntax—The Rust Book.](#)